Soil-Smart Planner — Build Spec for Claude Code

Nov 6, 2026 · @Ankit

How to use this spec

This is the build brief for v0.1 of Soil-Smart Planner, sized for a 48-hour hackathon. Everything after this section is written for Claude Code.

1. Export this doc as Markdown and save it as `SPEC.md` in an empty repo.
2. Open Claude Code in that repo and send:

```
Read SPEC.md. Build Milestone 1 only, run the tests, then stop and summarize
what you built and anything you assumed.
```

3. Review the result, then send the same message for Milestones 2, 3 and 4.

Building one milestone at a time keeps each step reviewable and leaves you with a working app after every stage.

Who does what

- **Builder:** runs Claude Code and reviews each milestone.
- **Research person:** replaces the placeholder values in `data/*.yaml` with numbers from one extension soil-test guide and sets `verified: true` on each row checked. No Python needed.
- **Users person:** checks the demo persona against what real growers say.

Goal and scope

Build a local web app that turns a small grower's plot data into a season plan for each plot: crop rotation, lime, feeding and pest prevention. Every line in the plan has a cost and a cited source. The product goal is less synthetic fertilizer and pesticide at about the same yield.

Two layers work together. A deterministic rules engine reads the data files and sets every number: crops, rates, amounts, costs. Claude, the AI layer, reasons on top of it: it explains the plan, answers questions, runs what-if scenarios through the engine and reads soil test reports. The pitch line: research sets the numbers, AI makes them useful.

In scope (v0.1)

- One grower with 1 to 5 plots, entered in a form or loaded from a demo file
- Rotation checks using crop families and 2 years of plot history
- Lime recommendation from soil pH
- N-P-K need from the crop and soil test, minus credits from compost and a previous legume
- Compost-first feeding plan, with a fertilizer top-up only for what compost doesn't cover
- Cost comparison: last season's inputs vs. the plan
- Pest prevention practices per crop (scouting, traps, row covers), never spray recommendations
- A "Why?" panel on every plan line: rule, values used, source, verified status
- Plan export as Markdown
- A Data Status page showing which data rows are verified

AI layer (in scope)

- Advisor notes: a short, prioritized note per plot on what matters most this season and what to watch
- Ask-your-plan chat that can run what-if scenarios through the engine ("what if I keep tomatoes in Plot A?")
- Smart intake: upload a soil test report or describe the garden in plain words, then review and confirm the extracted fields
- Guards so the AI never invents a number or a source, and a cache so the demo still works on bad Wi-Fi

If the plan costs more than last season, show that plainly. Organic inputs are not automatically cheaper, and the tool must not hide it.

Out of scope (do not build)

- User accounts, login, a database, cloud deployment
- Weather, irrigation, planting dates, labor, varieties, past-yield analysis
- Pesticide product or rate recommendations
- Automatic crop assignment across plots (optimization)
- AI that sets fertilizer, lime or pesticide amounts (the engine does that)
- A mobile app

Tech stack and project structure

- **Backend:** Python 3.11+, Flask, Pydantic v2 for models and validation, PyYAML for the data files, pytest, the Anthropic Python SDK for the AI layer, python-dotenv for local settings
- **Frontend:** React with Vite and TypeScript, plain CSS (no UI library, no router package)

Ask before adding any other Python or npm package.

```
soil-smart-planner/
├── SPEC.md
├── README.md
├── TODO.md                     # out-of-scope ideas noted during the build
├── .gitignore                  # includes backend/.env and
backend/.ai_cache/
├── backend/
│   ├── requirements.txt
│   ├── .env.example            # ANTHROPIC_API_KEY, LLM_MODEL, AI_CACHE
│   ├── app.py                  # Flask app: /api routes only
│   ├── engine/
│   │   ├── __init__.py         # exposes build_plan() and load_data()
│   │   ├── models.py           # Pydantic models: inputs, data rows, plan
│   │   ├── loader.py           # loads + validates data/*.yaml
│   │   ├── units.py            # area and rate conversions
│   │   ├── rotation.py
│   │   ├── lime.py
│   │   ├── nutrients.py
│   │   ├── credits.py
│   │   ├── feeding.py          # compost + top-up amounts and costs
│   │   ├── practices.py
│   │   ├── plan.py             # runs the pipeline, assembles the Plan
│   │   └── export.py           # Plan to Markdown
│   ├── ai/
│   │   ├── __init__.py
│   │   ├── client.py           # Anthropic SDK wrapper, tool loop, timeout,
cache
│   │   ├── models.py           # AdvisorNote, ChatReply, GrowerDraft
│   │   ├── tools.py            # tool schemas + handlers that call the
engine
│   │   ├── advisor.py
│   │   ├── chat.py
│   │   ├── intake.py
│   │   ├── guards.py           # number guard, citation guard
```

```
|   |   └── prompts/            # advisor.md, chat.md, intake.md
|   ├── data/
|   |   ├── sources.yaml
|   |   ├── source_notes.yaml   # research notes Claude searches
|   |   ├── crops.yaml
|   |   ├── problems.yaml       # pests and diseases
|   |   ├── rotation_rules.yaml
|   |   ├── lime_table.yaml
|   |   ├── soil_test_levels.yaml
|   |   ├── nutrient_recs.yaml
|   |   ├── credits.yaml
|   |   ├── products.yaml
|   |   └── practices.yaml
|   ├── fixtures/
|   |   └── demo_grower.json
|   └── tests/
|       ├── golden/             # guide worked examples, added by research
|       ├── fakes.py            # FakeClaudeClient with scripted replies
|       ├── test_units.py
|       ├── test_loader.py
|       ├── test_rotation.py
|       ├── test_lime.py
|       ├── test_nutrients.py
|       ├── test_credits.py
|       ├── test_feeding.py
|       ├── test_invariants.py
|       ├── test_golden.py
|       ├── test_api.py
|       ├── test_ai_guards.py
|       ├── test_ai_tools.py
|       └── test_ai_api.py
└── frontend/
    ├── package.json
    ├── vite.config.ts          # proxies /api to Flask on :5001
    ├── index.html
    └── src/
        ├── main.tsx
        ├── App.tsx             # tab bar: Planner, Data Status
        ├── api.ts              # every fetch call lives here
        ├── types.ts            # mirrors the Pydantic models
        ├── styles.css
        ├── pages/
        |   ├── PlannerPage.tsx
```

```
        │   └── DataStatusPage.tsx
        └── components/
            ├── GrowerForm.tsx
            ├── PlotForm.tsx
            ├── PlanSummary.tsx
            ├── PlotCard.tsx
            ├── WhyPanel.tsx
            ├── CostTable.tsx
            ├── AdvisorPanel.tsx
            ├── ChatPanel.tsx
            ├── ScenarioDiff.tsx
            └── IntakeDialog.tsx
```

Run in development, in two terminals:

```
cd backend && pip install -r requirements.txt && flask --app app run --port
5001 --debug
cd frontend && npm install && npm run dev
```

Vite proxies `/api` to `http://localhost:5001`, so no CORS setup is needed; port 5001 avoids the macOS AirPlay receiver on 5000. The engine never imports Flask. `app.py` only validates requests, calls `build_plan()` or `load_data()`, and returns JSON. The React app never computes agronomy.

Put `ANTHROPIC_API_KEY`, `LLM_MODEL` and `AI_CACHE` in `backend/.env`, which Flask loads through python-dotenv. Commit `.env.example`, never `.env` or the key.

Data model

Internally, every area is in square feet and every rate is in lb per 1,000 sq ft. Convert only at the edges: form input and display. 1 acre = 43,560 sq ft.

Inputs (`engine/models.py`)

Model	Fields
`Grower`	`name`, `plots: list[Plot]`, `crops_needed: list[str]` (crop ids), `compost_available_cu_yd: float`, `compost_price_usd_per_cu_yd: float` (default 0, for on-farm compost), `price_overrides: dict[str, float]` (product id to price)
`Plot`	`id`, `name`, `area_sqft: float`, `soil_texture` (sandy, loam or clay), `soil_test: SoilTest`, `crop_two_years_ago` (optional), `crop_last_year`, `crop_planned`, `problems: list[str]` (problem ids), `baseline: Baseline`
`SoilTest`	`ph: float`, `p_ppm: float`, `k_ppm: float`, `organic_matter_pct: float`
`Baseline`	`inputs: list[BaselineInput]` (last season), `pesticide_applications: int`
`BaselineInput`	`product_id`, `amount_lb: float`

Output

Model	Fields
`Plan`	`grower_name`, `plots: list[PlotPlan]`, `totals: CostSummary`, `warnings: list[str]`, `data_version`
`PlotPlan`	`plot_id`, `crop_planned`, `crop_suggested` (equals planned when no change), `lines: list[PlanLine]`, `baseline_cost_usd`, `plan_cost_usd`
`PlanLine`	`category` (rotation, lime, feeding, nutrient_warning, pest_prevention), `text`, `quantity` (optional), `unit` (optional), `cost_usd` (optional), `rule_id`, `source_id`, `verified: bool`, `inputs_used: dict` (values the rule read, shown in the Why panel)

Model	Fields
`CostSummary`	`baseline_usd`, `plan_usd`, `difference_usd`, `synthetic_n_lb_baseline`, `synthetic_n_lb_plan`, `pesticide_applications_baseline`

Every `PlanLine` has a `rule_id` and a `source_id`. A line's `verified` is true only when every data row it used is verified.

The React app mirrors these models in `frontend/src/types.ts`. JSON uses the same snake_case field names on both sides, and areas always travel in sq ft.

Data files

All agronomy numbers live in `data/*.yaml`, never in Python. The research person edits these files; the code only reads them.

Placeholder convention. Until the research person fills them in, every row uses rough placeholder values marked `# PLACEHOLDER`, with `verified: false` and `source: SRC-PLACEHOLDER`. Pick placeholders that make the demo produce non-zero, sensible-looking output. Never invent real-looking citations, page numbers or URLs. The app must run on placeholders and label them clearly.

Loader rules (`engine/loader.py`)

- Validate every file with Pydantic. On any error, fail at startup with the file, row id and field.
- Every `source` must exist in `sources.yaml`; every crop, family, problem and product id referenced must exist.
- Exactly one product per role: `topup_n`, `topup_p`, `topup_k`, `lime`.
- `data_version` = a short hash of all data files.

Crops to include: tomato, pepper, eggplant, potato (nightshade); cabbage, broccoli, collards, kale (brassica); squash, cucumber, watermelon (cucurbit); bush bean, southern pea, snap pea (legume); onion, garlic (allium); sweet corn (grass); lettuce (aster); carrot (carrot); okra (mallow); sweet potato (morning glory). Families are botanical facts; everything else in a crop row is a placeholder until verified.

Problems to include: early blight and tomato hornworm (nightshade); cabbage worms (brassica); squash bugs and squash vine borer (cucurbit); Mexican bean beetle (legume).

Schemas (one example entry each)

```
# sources.yaml
- id: SRC-PLACEHOLDER
  title: "Placeholder: replace with the extension guide used"
  publisher: ""
  year: null
  url: ""
  verified: false

# crops.yaml
- id: tomato
  name: Tomato
  family: nightshade
  n_demand: heavy              # heavy, medium, light or fixer
  target_ph: 6.5               # PLACEHOLDER
  n_need_lb_per_1000sqft: 2.0  # PLACEHOLDER
  source: SRC-PLACEHOLDER
  verified: false

# problems.yaml
- id: early-blight
  name: Early blight
  kind: disease                # pest or disease
  affects_families: [nightshade]
  source: SRC-PLACEHOLDER
  verified: false

# rotation_rules.yaml
- id: ROT-01
  kind: same_family_recent
  years_back: 2                # PLACEHOLDER; history covers 2 years, so max
2
  severity: warn
  message: "{crop} is in the same family ({family}) as a crop grown here in
the last {years_back} years."
  source: SRC-PLACEHOLDER
  verified: false
- id: ROT-02
  kind: prefer_after
  after_demand: heavy
  prefer_demand: fixer
  severity: suggest
```

```
  message: "A legume after a heavy feeder adds nitrogen for the next crop."
  source: SRC-PLACEHOLDER
  verified: false
- id: ROT-03
  kind: problem_history
  severity: strong_warn
  message: "{problem} was recorded on this plot; avoid {family} crops here
this season."
  source: SRC-PLACEHOLDER
  verified: false

# lime_table.yaml
# If the guide uses buffer pH, add a buffer_ph field and tell the builder.
- id: LIME-LOAM-01
  soil_texture: loam
  ph_min: 5.0                 # inclusive
  ph_max: 5.5                 # exclusive
  target_ph: 6.5
  lime_lb_per_1000sqft: 50    # PLACEHOLDER
  source: SRC-PLACEHOLDER
  verified: false

# soil_test_levels.yaml (thresholds depend on the lab's extraction method)
- id: STL-P
  nutrient: P
  extraction_method: "PLACEHOLDER, must match the guide"
  upper_bounds_ppm:           # PLACEHOLDER; above "high" = very_high
    very_low: 10
    low: 20
    medium: 40
    high: 80
  source: SRC-PLACEHOLDER
  verified: false

# nutrient_recs.yaml (placeholders must fall as the level rises)
- id: NUT-P2O5-HEAVY
  nutrient: P2O5
  n_demand: heavy
  lb_per_1000sqft:            # PLACEHOLDER
    very_low: 4
    low: 3
    medium: 2
    high: 0
```

```
    very_high: 0
  source: SRC-PLACEHOLDER
  verified: false

# credits.yaml
- id: CRED-COMPOST
  kind: compost
  n_lb_available_per_cu_yd: 3  # PLACEHOLDER, first-year plant-available N
  p2o5_lb_per_cu_yd: 2         # PLACEHOLDER
  k2o_lb_per_cu_yd: 2          # PLACEHOLDER
  max_cu_yd_per_1000sqft: 1    # PLACEHOLDER, cap on compost rate
  source: SRC-PLACEHOLDER
  verified: false
- id: CRED-LEGUME
  kind: previous_legume
  n_lb_per_1000sqft: 0.5       # PLACEHOLDER
  source: SRC-PLACEHOLDER
  verified: false

# products.yaml (analysis % from the label; prices are PLACEHOLDERS)
- id: fert-10-10-10
  name: "10-10-10 granular"
  type: synthetic
  role: null                   # baseline only
  n_pct: 10
  p2o5_pct: 10
  k2o_pct: 10
  bag_lb: 50
  price_usd_per_bag: 20        # PLACEHOLDER
  verified: false
# Also add: urea 46-0-0 (role topup_n), triple superphosphate 0-46-0
# (role topup_p), muriate of potash 0-0-60 (role topup_k), ag lime (role
lime).

# practices.yaml
- id: PRAC-SCOUT-HORNWORM
  crops: [tomato, pepper, eggplant]
  problems: [tomato-hornworm]  # empty list = applies to every listed crop
  text: "Scout weekly for hornworms and hand-pick before considering any
spray."
  source: SRC-PLACEHOLDER
  verified: false
```

Engine pipeline

`build_plan(grower, data) -> Plan` runs these steps in a fixed order. Every step is a pure function: no I/O, no randomness, no globals. The same input always gives the same output in the same line order.

1. **Normalize** (`units.py`): convert plot areas to sq ft.
2. **Rotation** (`rotation.py`): look up the families of the crops grown two years ago, last year, and planned.
   - ROT-01: the planned crop's family appears in the last `years_back` years of history → warn.
   - ROT-03: a recorded problem on the plot affects the planned crop's family → strong warn.
   - If warned, list alternatives from `crops_needed` whose family is not in this plot's history and not affected by its problems. Sort legumes first when last year's crop was a heavy feeder (ROT-02), then alphabetically by crop name.
   - Set `crop_suggested` to the first alternative (or keep the planned crop if there is none, with a warning). Keep `crop_planned` unchanged. Every later step uses `crop_suggested` .
3. **Lime** (`lime.py`): if pH is below the crop's target pH, find the `lime_table` row matching texture, pH band and target pH, then multiply by area. If pH is at or above target, add a "No lime needed" line. If no row matches, add a warning line, never a guessed number.
4. **Nutrient need** (`nutrients.py`): N need = the crop's rate × area. Classify soil P and K with `soil_test_levels` , then look up P2O5 and K2O recommendations by level and the crop's `n_demand` group × area.
5. **Credits** (`credits.py`): if last year's crop was a legume, subtract the legume N credit. Allocate the grower's compost across plots in descending order of N need, each plot up to the compost cap, until it runs out. Subtract compost N, P2O5 and K2O. Floor every remaining need at 0.
6. **Feeding** (`feeding.py`): add a compost line per plot. Convert each remaining nutrient need into its top-up product: amount = need ÷ analysis %. Price each line; compost costs `compost_price_usd_per_cu_yd` .
7. **Nutrient warnings**: if last season's inputs added P2O5 to a plot whose P level is high or very high, add a warning with the lb of P2O5 added. Do the same for K2O.

8. **Practices** (`practices.py`): add the practices that match the suggested crop and the plot's problems.
9. **Costs and totals** (`plan.py`): baseline cost = last season's inputs at current prices. Plan cost = lime + feeding lines. Fill `CostSummary` , including synthetic N for both, then check that plot totals add up to the grower totals.

Round only for display: lb to 0.1, dollars to cents. Keep full precision internally.

Backend API (Flask)

Every route lives under `/api` and returns JSON shaped by the Pydantic models. Data files load once at startup; if validation fails, the server refuses to start and prints the file, row and field.

Route	Request	Response
GET `/api/health`	none	`{status, data_version}`
GET `/api/reference`	none	Crops, problems and products for the form dropdowns, plus `data_version` and `unverified_count`
GET `/api/demo-grower`	none	The demo `Grower` from `fixtures/demo_grower.json`
POST `/api/plan`	a `Grower`	a `Plan`
POST `/api/plan/markdown`	a `Grower`	The plan as `text/markdown` , with citations and unverified tags
GET `/api/data-status`	none	Every data row (file, id, verified, source), counts per file, loader warnings

Invalid input returns 422 with `{errors: [{field, message}]}` built from Pydantic's errors. Anything unexpected returns 500 with `{error}` and logs the traceback. No auth, database or sessions.

AI layer (Claude)

Claude makes the planner smarter without making it less trustworthy. The engine stays the single source of every number, and Claude gets the engine as tools, so it reasons, compares and explains from real results instead of guessing.

Setup

- Anthropic Python SDK, Messages API with tool use. All AI code lives in `backend/ai/`.
- Model from the `LLM_MODEL` environment variable. Default `claude-sonnet-5-5` for speed; set `claude-opus-5-5` for the most capable answers if its latency is acceptable in the demo.
- `ANTHROPIC_API_KEY` lives only in the backend environment. With no key, `/api/ai/status` returns `enabled: false`, the AI features are hidden, and the rest of the app works unchanged.
- System prompts live in `backend/ai/prompts/*.md`, so the team can tune them without touching code.
- Limits per request: at most 6 tool-call rounds, a 45-second timeout, `max_tokens` around 1,500.

Tools Claude can call (`backend/ai/tools.py`, each a thin wrapper around the engine)

Tool	Input	Returns
`get_plan`	none	The current `Plan` from the engine
`run_scenario`	`changes: [{plot_id, field, value}]`, `keep_planned_crop_for: [plot_id]`	The new `Plan` plus a diff vs. the current one: cost change and lines added, removed or changed
`explain_rule`	`rule_id`	The rule row, its source row and verified status
`search_sources`	`query`	Top 5 entries from `data/source_notes.yaml`: id, source_id, note, page
`list_reference`	`kind`: crops, problems or products	Ids and names, so Claude only uses valid ids

`run_scenario` accepts only these fields: `crop_planned`, `area_sqft`, `soil_test.*`, `compost_available_cu_yd`, `price_overrides`. Validate with Pydantic and reject anything else. Add an optional `keep_planned_crop_for` argument to `build_plan()`: for those plots, rotation still warns, but `crop_suggested` stays the planned crop.

`data/source_notes.yaml` is Claude's reading material: short notes the research person writes in their own words, each tied to a source and page. Simple keyword scoring is enough for 15 to 20 sources, so no vector database.

```
- id: NOTE-001
  source: SRC-PLACEHOLDER
  page: ""
  topics: [rotation, nightshade, early-blight]
  note: "Placeholder: one or two sentences on the finding, in your own
words."
  verified: false
```

Feature 1: advisor notes (`POST /api/ai/advise`)

After the plan is built, Claude reviews it with the tools and returns one `AdvisorNote` per plot: `summary` (2 to 3 sentences), `priorities` (up to 3), `watch_for`, `trade_offs` and `source_ids`. Ask for JSON, validate it with Pydantic, retry once on invalid output, then fall back to a plain template note built from the plan lines.

Feature 2: ask-your-plan chat (`POST /api/ai/chat`)

The chat is stateless: the frontend sends the grower and the full message history each time. Claude answers in plain language, cites source ids, and calls `run_scenario` for any what-if question. When it ran a scenario, the response includes it, so the UI can show the diff and offer "Apply this change". For anything the data doesn't cover, Claude says so and gives general guidance with no rates, labeled as not from verified rules.

Feature 3: smart intake (`POST /api/ai/intake`)

Accepts a PDF or photo of a soil test report, or free text such as "a 20 by 30 ft bed where I grew tomatoes last year". Claude returns a `GrowerDraft` with the fields it found plus `uncertain_fields`. The user reviews and confirms in the UI; intake never fills the form or builds a plan on its own.

Guards (`backend/ai/guards.py`, applied to every AI response)

- Number guard: every number in AI text must appear in a tool result or in the grower's input for this request, after rounding to display precision. Integers 0 to 12 are allowed for ordinary wording. On a violation, retry once and tell Claude which numbers failed, then use the fallback.
- Citation guard: a source id must exist in `sources.yaml` and must have come back from a tool in this request. Drop anything else.
- Claude never sets a rate, amount or cost, and never recommends a pesticide product or rate. The system prompt says so; the guards enforce the numbers part.
- AI text is always labeled "AI advisor" in the UI, separate from the engine's plan lines.

Demo reliability

With `AI_CACHE=1`, cache every AI response in `backend/.ai_cache/`, keyed by a hash of the endpoint, request body, model and `data_version`. Warm the cache with the demo persona and the demo questions before the pitch. If a live call fails and a cached answer exists, serve it with a small "cached" label; otherwise show a friendly error and keep the plan on screen.

Demo questions to prepare (with the demo persona)

- "What if I keep tomatoes in Plot A?" → `run_scenario` with `keep_planned_crop_for`; Claude explains the rotation and early blight risk.
- "Why no phosphorus on Plot C?" → `explain_rule` and `search_sources`; Claude points to the soil test level and last season's inputs.
- "What if I only have 1 cubic yard of compost?" → `run_scenario` on `compost_available_cu_yd`; Claude explains the change in cost and synthetic N using the diff.

Endpoints

Route	Request	Response
GET `/api/ai/status`	none	`{enabled, model}`
POST `/api/ai/advise`	`{grower}`	`{notes: [AdvisorNote], cached}`
POST `/api/ai/chat`	`{grower, messages: [{role, content}]}`	`{reply, source_ids, scenario (plan and diff, or null), cached}`

Route	Request	Response
POST `/api/ai/intake`	multipart `file`, or `{text}`	`{grower_draft, uncertain_fields}`

When AI is disabled, these return 503 with `{error: "AI disabled"}`.

Frontend (React)

Two views, switched by a simple tab bar in `App.tsx`: Planner and Data Status. Plain CSS in `styles.css`, readable on a laptop and a phone. Every fetch call lives in `api.ts`; components never call `fetch` directly.

Planner view

- Header bar: "Load demo grower", "Upload grower JSON" and "Download grower JSON" (both handled in the browser), a unit toggle (sq ft or acres), the data version, and the count of unverified data rows. The unit toggle only changes input and display; the API always gets sq ft.
- Grower form: name, crops needed (multi-select), compost available (cu yd), compost price, and product price overrides in a collapsed section.
- Plots: one collapsible section per plot, with "Add plot" and "Remove", up to 5 plots. Crops, problems and products are dropdowns filled from `/api/reference`, never free text.
- "Build plan" posts the grower to `/api/plan`. Show a loading state, and show 422 field errors next to the matching inputs.

Results

- Summary row (`PlanSummary`): baseline cost, plan cost, difference, and synthetic N in lb (baseline → plan). If the plan costs more, show it plainly.
- One card per plot (`PlotCard`). Header: plot name, size, and planned → suggested crop, e.g. "Plot A · 0.25 acre · Tomato → Bush bean". Lines are grouped as Rotation, Soil, Feeding, Warnings and Pest prevention. Each line shows its text, quantity, unit and cost, plus an "unverified" tag when `verified` is false.
- A "Why?" toggle on every line (`WhyPanel`): rule id, rule message, the values used, source title and link, and verified status.
- A cost comparison table (`CostTable`): one row per plot plus a total, with baseline $, plan $ and difference $.

- "Download plan (Markdown)" calls `/api/plan/markdown` and saves the file.
- Under the results: "Planning support, not professional agronomic advice. Check soil tests and recommendations with your local extension service."

AI features (shown only when `/api/ai/status` reports enabled)

- Advisor notes: after a plan builds, call `/api/ai/advise` and show an `AdvisorPanel` on each plot card, visually distinct and labeled "AI advisor". Source ids show as chips that open the source in the Why panel.
- Chat: a side drawer (`ChatPanel`) with the three demo questions as suggestions. When a reply includes a scenario, show `ScenarioDiff` (cost before → after, lines changed) with an "Apply this change" button that updates the form and rebuilds the plan.
- Smart intake: an "Import soil test or describe your garden" button opens `IntakeDialog`. The user uploads a file or pastes text, reviews the extracted fields with uncertain ones highlighted, then "Use these values" fills the form.
- Every AI call has a loading state. On an error, show a friendly message and keep the plan visible. Show a "cached" label when the backend served a cached answer.
- One line near the AI features: "AI features send your plot data to the Claude API."

Data Status view

- A table from `/api/data-status`: file, id, verified, source, with verified / total counts per file and any loader warnings.

Demo persona

`fixtures/demo_grower.json` is a fictional small grower with three plots, each built to show off one part of the engine. These are input values, not agronomy claims.

Grower: "Demo Market Garden". Crops needed: tomato, bush bean, cabbage, squash, onion. Compost available: 3 cu yd at $0 (on-farm).

Plot	Size, texture	Soil test: pH, P ppm, K ppm, OM %	Two years ago → last year → planned	Problems	Last season
A	0.25 acre, loam	6.4, 45, 120, 2.5	pepper → tomato → tomato	early blight	250 lb 10-10-10, 6 sprays
B	2,500 sq ft, sandy	5.4, 18, 60, 1.5	tomato → bush bean → cabbage	cabbage worms	50 lb 10-10-10, 4 sprays

Plot	Size, texture	Soil test: pH, P ppm, K ppm, OM %	Two years ago → last year → planned	Problems	Last season
C	1,000 sq ft, clay	6.8, 110, 200, 4.0	bush bean → squash → squash	squash bugs	30 lb 10-10-10, 5 sprays

What the demo should show (check after M2 and M3)

Plot	Expected
A	Tomato flagged by ROT-01 and ROT-03. Suggested crop: bush bean, legume first after a heavy feeder (ROT-02).
B	No rotation flag. Legume N credit applied. Lime recommended, since pH 5.4 is below the target.
C	Squash flagged by ROT-01 and ROT-03. Suggested crop: cabbage. No P2O5 recommended, plus a warning that last season added 3 lb of P2O5 to a plot whose P is very high.
Totals	Plan synthetic N is below the baseline's 33 lb.

If the placeholder data breaks one of these, fix the placeholder values, not the engine. After the research person replaces the placeholders, re-run the demo; if a plot stops telling its story (for example, 110 ppm is no longer "very high" under the real thresholds), adjust the persona's soil values, never the tables.

Tests

Run `pytest` in `backend/` and `npm run build` in `frontend/` after every milestone. Both must pass before moving on.

Unit tests

- Units: sq ft and acre conversions round-trip.
- Rotation: same family flagged; problem history flagged; legumes first after a heavy feeder; alphabetical tie-break; no alternatives → warning.
- Lime: pH exactly on a band boundary; pH at target → "No lime needed"; no matching row → warning, not a number.
- Soil test levels: values exactly on each threshold.

- Credits: need never goes below 0; compost allocated never exceeds what the grower has or the per-plot cap.
- Feeding: amount = need ÷ analysis %; costs use price overrides when set.

Invariants (`test_invariants.py` , on the demo persona plus 3 small hand-written growers)

- Every `PlanLine` has a `rule_id` and a `source_id` that exist in the data.
- No negative quantities or costs.
- `verified` is false whenever any data row a line used is unverified.
- Building the same plan twice gives identical output, including line order.
- Total compost allocated ⩽ compost available.
- Grower totals equal the sum of plot totals.

Golden cases (`test_golden.py` + `tests/golden/*.json`)

Each case is an input plus the expected numbers from a worked example in the extension guide. Build the scaffold with one example case marked `pytest.mark.skip(reason="awaiting verified data")` . The research person adds real cases, which gives the team a pitch line: "matches the guide on N of N test cases."

API and frontend

- `test_api.py` (Flask test client): `/api/plan` returns a valid `Plan` for the demo grower; invalid input returns 422 with field errors; `/api/data-status` counts match the data files.
- `npm run build` passes with no TypeScript errors.

AI tests (all with `FakeClaudeClient` from `tests/fakes.py` ; no test calls the real API)

- Guards: an invented number is caught, triggers one retry, then the fallback; numbers from tool results and grower input pass; unknown or unreturned source ids are dropped.
- Tools: `run_scenario` rejects fields outside the allowed list, returns a correct diff, and `keep_planned_crop_for` keeps the planned crop while still warning.
- Endpoints: with no API key, `/api/ai/status` reports disabled and the AI routes return 503; advise and chat responses validate against their models; the cache returns the same answer for the same request.

Milestones

Build in this order. After each one, run the tests, stop, and summarize what changed and anything you assumed.

#	Build	Done when	Target
M1	Backend and frontend scaffolds, models, loader, placeholder data for every listed crop and problem, demo fixture, `/api/health`, `/api/reference`, `/api/demo-grower`, a stub `/api/plan` with one dummy line per plot, Planner form with "Load demo grower"	Both servers start with the commands above; the demo loads into the form; "Build plan" shows the stub lines; loader and unit tests pass	Hour 6
M2	Rotation, lime, nutrient need, credits and practices behind the real `/api/plan`; plot cards in the UI	The demo shows the expected story for plots A, B and C (Demo persona section); unit and API tests pass	Hour 14
M3	Feeding amounts, costs, cost comparison, Why panels, unverified tags, Markdown export, Data Status view	Summary and per-plot costs show; every line opens a Why panel; the export downloads with citations	Hour 20
M4	AI layer core: client with tool loop, tools, guards, cache, `/api/ai/status`, `/api/ai/advise`, `/api/ai/chat` with `run_scenario`, `AdvisorPanel`, `ChatPanel`, `ScenarioDiff`, 3 to 5 placeholder source notes	The three demo questions get guarded, cited answers; AI tests pass with the fake client; with no API key the app still works and hides AI	Hour 30
M5	Smart intake, invariant tests, golden-case scaffold, README (run both servers, set the API key, add a crop, add a rule, add a source note, mark a row verified), warm the demo cache, UI polish	All tests pass and `npm run build` succeeds; a soil test PDF or photo becomes a reviewable draft; the demo runs from cache with Wi-Fi off	Hour 36

The hours count from the start of the hackathon. The rest of the time goes to verifying data, the deck and rehearsal.

Rules for Claude Code

- Work one milestone at a time. Don't start the next until asked.
- No agronomy numbers in Python. If you need a number, add it to a data file with `verified: false`.
- Never invent sources, page numbers or URLs. Use `SRC-PLACEHOLDER` until the team provides real ones.
- No LLM or network calls in the engine; they all live in backend/ai. With AI off or offline, the planner still works.
- Keep engine functions pure and deterministic. The React app never computes agronomy; it only shows what the API returns.
- When a lookup has no matching row, return a warning line, not a guessed or default value.
- Stay inside the scope list. If something seems needed but is out of scope, note it in `TODO.md` and keep going.
- Ask before adding a Python or npm package, or before changing a model in a way that breaks the demo fixture.
- Make error messages name the file, row and field.
- Commit after each milestone with a message like `M2: rotation, lime, nutrients, credits`.
- The AI never produces a quantity, rate or cost that didn't come from an engine tool result, and never names a pesticide product or rate.
- Tests never call the real API; use the fake client in `tests/fakes.py`.

Stretch (only if everything above is done)

- Pest photo help in the chat: Claude describes what it sees and what to scout for, labeled as uncertain, and suggests confirming with the extension service. No treatment rates.
- Plans in Spanish: Claude translates the explanations and advisor notes; the engine's numbers and units stay untouched.
- Streaming chat replies for a snappier demo.